Task 1

In [73]:
```
'''Enviironment Setup'''

# Required libraries
import tensorflow as tf
import numpy as np
import random
import matplotlib.pyplot as plt
import pandas as pd

# Setting seeds for reproducibility
tf.random.set_seed(42)
np.random.seed(42)
random.seed(42)

# Versions
print("TensorFlow Version:", tf.__version__)
print("NumPy Version:", np.__version__)
print("Matplotlib Version:", plt.matplotlib.__version__)
print("Pandas Version:", pd.__version__)

# GPU availability
gpus = tf.config.list_physical_devices('GPU')
if gpus:
    print("GPU is available:", gpus)
else:
    print("GPU is NOT available. Running on CPU.")
```

```
TensorFlow Version: 2.19.0
NumPy Version: 2.1.3
Matplotlib Version: 3.10.1
Pandas Version: 2.2.3
GPU is NOT available. Running on CPU.
```

The reason that CPU is slower than GPU, is that CPU operates on an operation by operation (sequentially) and only has a small number of core processors, while a GPU can perform parallel computations on thousands of cores making it much better suited to creating large, highly complex matrices during the training of deep learning models. If I were to have a GPU available to me, I would be able to create larger batch sizes, and train deeper models at a much faster rate.

Random seeds are:

- Random seed for Tensorflow; controls random weights and layers in models;
- Random Seed for Numpy; controls randomness associated with data set operation;
- Random Seed for Python; contains overall level of randomness.

Establishing all of these random seeds will result in reproducible outcomes.

In [8]:
```
'''Dataset Exploration'''
```

```
from tensorflow.keras.datasets import mnist, cifar10

# Load datasets
(x_train_mnist, y_train_mnist), (x_test_mnist, y_test_mnist) = mnist.load
(x_train_cifar, y_train_cifar), (x_test_cifar, y_test_cifar) = cifar10.lo
```

In [9]:
```
# MNIST
print("MNIST Train Shape:", x_train_mnist.shape)
print("MNIST Test Shape:", x_test_mnist.shape)

# CIFAR-10
print("CIFAR Train Shape:", x_train_cifar.shape)
print("CIFAR Test Shape:", x_test_cifar.shape)

# Data type and range
print("\nMNIST dtype:", x_train_mnist.dtype)
print("MNIST range:", x_train_mnist.min(), "to", x_train_mnist.max())

print("\nCIFAR dtype:", x_train_cifar.dtype)
print("CIFAR range:", x_train_cifar.min(), "to", x_train_cifar.max())
```

```
MNIST Train Shape: (60000, 28, 28)
MNIST Test Shape: (10000, 28, 28)
CIFAR Train Shape: (50000, 32, 32, 3)
CIFAR Test Shape: (10000, 32, 32, 3)

MNIST dtype: uint8
MNIST range: 0 to 255

CIFAR dtype: uint8
CIFAR range: 0 to 255
```

In [10]:
```
unique, counts = np.unique(y_train_mnist, return_counts=True)

print("\nMNIST Class Distribution:")
for u, c in zip(unique, counts):
    print(f"Class {u}: {c} samples")
```

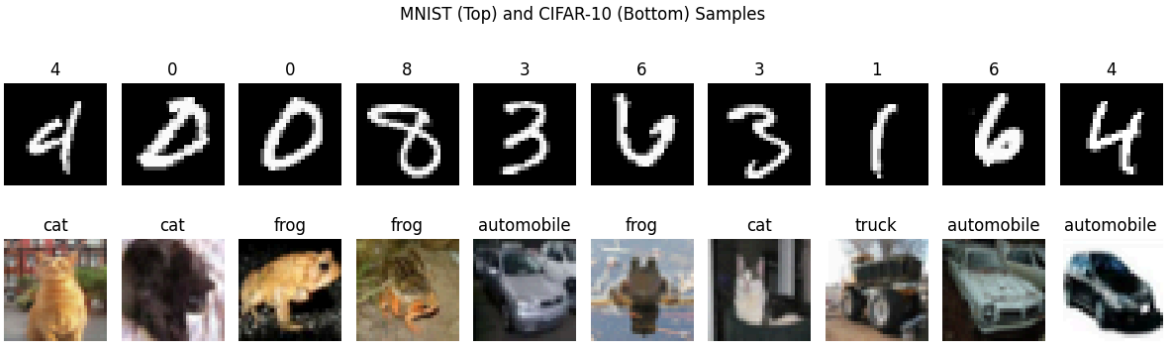
```
MNIST Class Distribution:
Class 0: 5923 samples
Class 1: 6742 samples
Class 2: 5958 samples
Class 3: 6131 samples
Class 4: 5842 samples
Class 5: 5421 samples
Class 6: 5918 samples
Class 7: 6265 samples
Class 8: 5851 samples
Class 9: 5949 samples
```

MNIST dataset is approximately balanced as each class has nearly equal samples.

In [11]:
```
plt.figure(figsize=(12, 4))

# MNIST samples
for i in range(10):
    idx = np.random.randint(0, len(x_train_mnist))
    plt.subplot(2, 10, i + 1)
    plt.imshow(x_train_mnist[idx], cmap='gray')
    plt.title(y_train_mnist[idx])
```

```
    plt.axis('off')

# CIFAR samples
class_names = ['airplane','automobile','bird','cat','deer',
               'dog','frog','horse','ship','truck']

for i in range(10):
    idx = np.random.randint(0, len(x_train_cifar))
    plt.subplot(2, 10, i + 11)
    plt.imshow(x_train_cifar[idx])
    plt.title(class_names[int(y_train_cifar[idx])])
    plt.axis('off')

plt.suptitle("MNIST (Top) and CIFAR-10 (Bottom) Samples")
plt.tight_layout()
plt.savefig("dataset_samples.png")
plt.show()
```

```
/var/folders/s8/fcvrqrkx4m54_c385ys4y3k80000gn/T/ipykernel_7741/91680008.py:19: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
  plt.title(class_names[int(y_train_cifar[idx])])
```



In [12]:
```
''' Preprocessing Pipeline '''

from tensorflow.keras.utils import to_categorical

def preprocess_data(x, y, is_mnist=True):
    print("\nBefore preprocessing:")
    print("Shape:", x.shape, "Dtype:", x.dtype,
          "Min:", x.min(), "Max:", x.max())

    # Normalize
    x = x.astype('float32') / 255.0

    # Reshape MNIST
    if is_mnist:
        x = x.reshape(-1, 28, 28, 1)

    # One-hot encode
    y = to_categorical(y, 10)

    print("\nAfter preprocessing:")
    print("Shape:", x.shape, "Dtype:", x.dtype,
          "Min:", x.min(), "Max:", x.max())

    return x, y
```

In [13]:
```
x_train_mnist, y_train_mnist = preprocess_data(x_train_mnist, y_train_mni
x_test_mnist, y_test_mnist = preprocess_data(x_test_mnist, y_test_mnist,

x_train_cifar, y_train_cifar = preprocess_data(x_train_cifar, y_train_cif
x_test_cifar, y_test_cifar = preprocess_data(x_test_cifar, y_test_cifar,
```

```
Before preprocessing:
Shape: (60000, 28, 28) Dtype: uint8 Min: 0 Max: 255

After preprocessing:
Shape: (60000, 28, 28, 1) Dtype: float32 Min: 0.0 Max: 1.0

Before preprocessing:
Shape: (10000, 28, 28) Dtype: uint8 Min: 0 Max: 255

After preprocessing:
Shape: (10000, 28, 28, 1) Dtype: float32 Min: 0.0 Max: 1.0

Before preprocessing:
Shape: (50000, 32, 32, 3) Dtype: uint8 Min: 0 Max: 255

After preprocessing:
Shape: (50000, 32, 32, 3) Dtype: float32 Min: 0.0 Max: 1.0

Before preprocessing:
Shape: (10000, 32, 32, 3) Dtype: uint8 Min: 0 Max: 255

After preprocessing:
Shape: (10000, 32, 32, 3) Dtype: float32 Min: 0.0 Max: 1.0
```

The process of normalization consists of taking the pixel values and dividing them by the number 255.0, resulting in pixel values within the range [0,1].

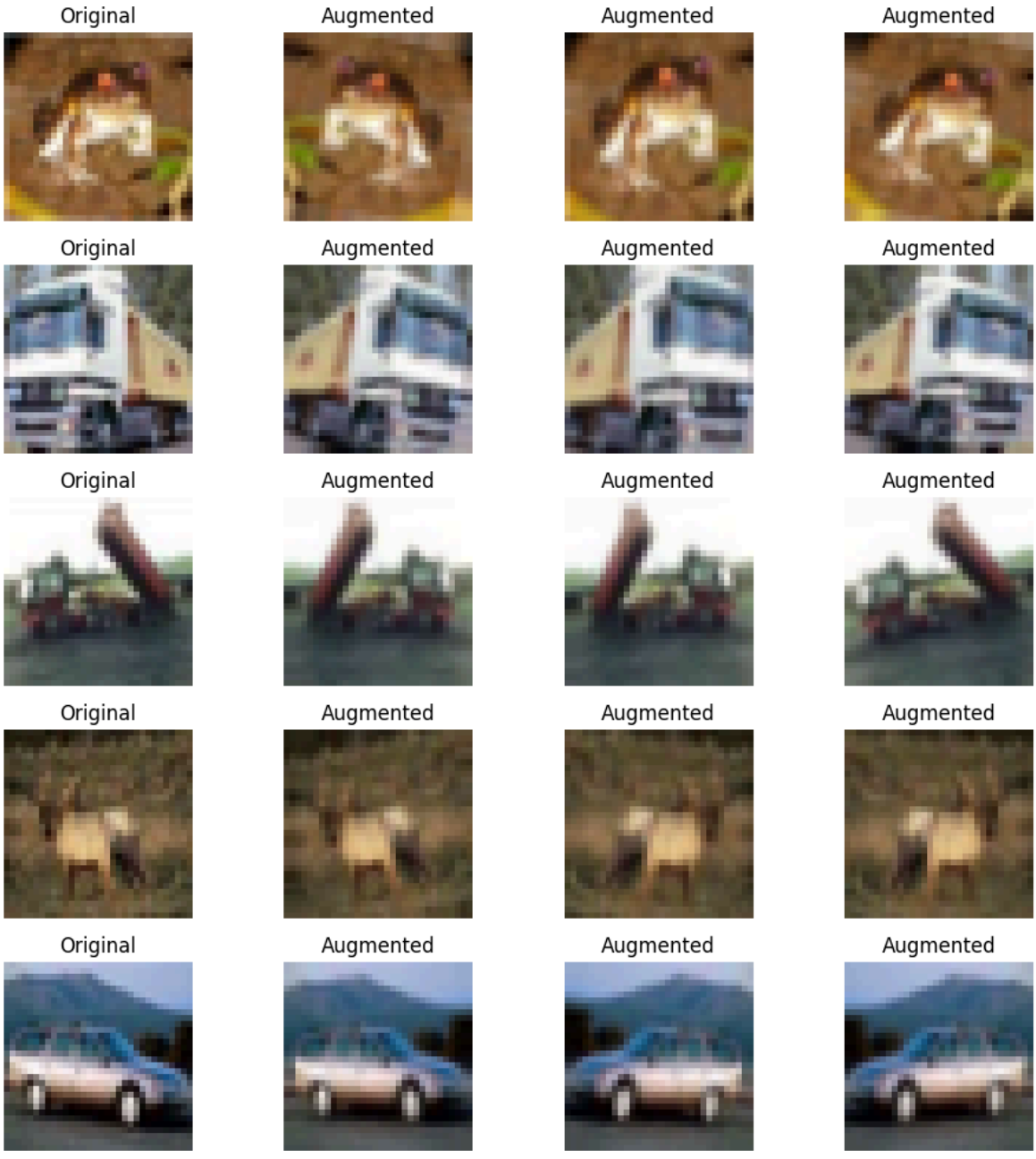
When using the number 255.0 for normalizing purposes floating-point will always be used. If the number 255 was to be used as an integer then integer division would produce incorrect results as well.

In order to have the MNIST images made compatible with CNN input format they must be reshaped with a channel dimension of 1.

In [14]:
```
''' Data Augmentation '''

from tensorflow.keras.preprocessing.image import ImageDataGenerator

datagen = ImageDataGenerator(
    horizontal_flip=True,
    rotation_range=10,
    zoom_range=0.1
)
```

In [15]:
```
plt.figure(figsize=(10, 10))

for i in range(5):
    img = x_train_cifar[i]

    # Original
    plt.subplot(5, 4, i*4 + 1)
    plt.imshow(img)
```

```
    plt.title("Original")
    plt.axis('off')

    # Augmented versions
    img = img.reshape((1,) + img.shape)
    aug_iter = datagen.flow(img, batch_size=1)

    for j in range(3):
        aug_img = next(aug_iter)[0]
        plt.subplot(5, 4, i*4 + j + 2)
        plt.imshow(aug_img)
        plt.title("Augmented")
        plt.axis('off')

plt.tight_layout()
plt.savefig("augmentation_demo.png")
plt.show()
```



Data augmentation will only enhance the training data since it artificially creates new diverse pieces of data to use to train your network.

If you apply augmentation to your validation or test data, you will change the original distribution of the data and produce invalid evaluations and inaccurate performance metrics.

Q1. What does the channel dimension represent in a tensor of shape (N, H, W, C)? Explain with reference to both a greyscale image and an RGB image.

Answer: The channel dimension indicates how many features each pixel in the image has. The number of channels in an image is given by C. Grayscale images have C = 1 (single intensity). RGB images will have C = 3 (one Red channel, one Green channel and one Blue channel).

Q2. CIFAR-10 images are 32×32 pixels. If you were training a CNN on 1024×1024 satellite images, what data loading strategies would you use to avoid running out of memory? Name at least two techniques and explain how each one helps.

Answer:

1. Loading in Batches: Load Data in smaller Parts than all at once
2. Changing Size of Images: Make image size smaller which will require less memory
3. Using Data Generators: Create new image data (load it into RAM) each time during Training as opposed to loading every image from the entire dataset into RAM together.

Q3. Suppose a student applies normalisation to the test set using the mean and standard deviation computed from the test set itself. What is wrong with this approach?

Answer: This method is incorrect due to the fact that your test set must be kept completely separate from your training set during testing. The test set data must not come in contact with any statistics provided by the training set which can happen very easily; this will cause data leaking and give you incorrect results when performing the analysis. It is critical that any normalizing factors are calculated exclusively on the training set's data to avoid this

Task 2

In [19]:
```
''' Manual 2D Convolution (NumPy only) '''

import numpy as np

def conv2d(image, kernel, stride=1, padding=0):
    # Get dimensions
    img_h, img_w = image.shape
    k_h, k_w = kernel.shape

    # Add padding
    if padding > 0:
```

```
        image = np.pad(image, ((padding, padding), (padding, padding)), m

    # Output dimensions
    out_h = (image.shape[0] - k_h) // stride + 1
    out_w = (image.shape[1] - k_w) // stride + 1

    output = np.zeros((out_h, out_w))

    # Convolution operation
    for i in range(out_h):
        for j in range(out_w):
            region = image[i*stride:i*stride+k_h, j*stride:j*stride+k_w]
            output[i, j] = np.sum(region * kernel)

    return output
```

In [20]:
```
# Given image
image = np.array([
    [3,1,0,2,4],
    [1,5,3,2,1],
    [0,2,6,4,3],
    [2,3,1,5,2],
    [1,0,2,3,4]
])

# Sobel-X kernel
kernel = np.array([
    [-1,0,1],
    [-2,0,2],
    [-1,0,1]
])

output = conv2d(image, kernel, stride=1, padding=0)

print("Output Feature Map:\n", output)
print("Output Shape:", output.shape)
```

```
Output Feature Map:
 [[ 7. -3. -3.]
 [13.  3. -7.]
 [ 5.  9.  1.]]
Output Shape: (3, 3)
```

Question (a) Input: 28×28, Kernel: 5×5, Padding: 0 (valid), Stride: 1

Answer: Input = 28, Kernel = 5, Padding = 0, Stride = 1

Output = (28 - 5 + 0)/1 + 1 = 24

Final size = 24 × 24

Question (b) Input: 28×28, Kernel: 3×3, Padding: 1 (same), Stride: 1

Answer: Input = 28, Kernel = 3, Padding = 1, Stride = 1

Output = (28 - 3 + 2)/1 + 1 = 28

Final size = 28 × 28

Question (c) Input: 32×32, Kernel: 3×3, Padding: 0 (valid), Stride: 2

Answer:

Input = 32, Kernel = 3, Padding = 0, Stride = 2

Output = floor((32 - 3)/2) + 1 = floor(29/2) + 1 = 14 + 1 = 15

Final size = 15 × 15

Question (d) Two consecutive Conv2D layers: first with K=3, P=1, S=1 applied to 32×32; then K=3, P=0, S=1 applied to the output. What is the final size?

Answer: First layer: (32 - 3 + 2)/1 + 1 = 32

Second layer: (32 - 3 + 0)/1 + 1 = 30

Final size = 30 × 30

In [21]:
```
''' Implement LeNet-5 '''
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, AveragePooling2D, Flatten, De

model = Sequential()

# Layer 1
model.add(Conv2D(6, (5,5), input_shape=(28,28,1), padding='valid'))
model.add(Activation('tanh'))
model.add(AveragePooling2D(pool_size=(2,2), strides=2))

# Layer 2
model.add(Conv2D(16, (5,5), padding='valid'))
model.add(Activation('tanh'))
model.add(AveragePooling2D(pool_size=(2,2), strides=2))

# Fully connected
model.add(Flatten())
model.add(Dense(120))
model.add(Activation('tanh'))

model.add(Dense(84))
model.add(Activation('tanh'))

model.add(Dense(10, activation='softmax'))

# Summary
model.summary()
```

```
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-pac
kages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequenti
al models, prefer using an `Input(shape)` object as the first layer in the
model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	
conv2d (Conv2D)	(None, 24, 24, 6)	
activation (Activation)	(None, 24, 24, 6)	
average_pooling2d (AveragePooling2D)	(None, 12, 12, 6)	
conv2d_1 (Conv2D)	(None, 8, 8, 16)	
activation_1 (Activation)	(None, 8, 8, 16)	
average_pooling2d_1 (AveragePooling2D)	(None, 4, 4, 16)	
flatten (Flatten)	(None, 256)	
dense (Dense)	(None, 120)	
activation_2 (Activation)	(None, 120)	
dense_1 (Dense)	(None, 84)	
activation_3 (Activation)	(None, 84)	
dense_2 (Dense)	(None, 10)	

Total params: 44,426 (173.54 KB)
Trainable params: 44,426 (173.54 KB)
Non-trainable params: 0 (0.00 B)

Question (b) Manually calculate the parameter count for the first Conv2D layer only. Show the formula: (K×K×C_in + 1) × C_out.

Answer:

```
Formula:
(K × K × C_in + 1) × C_out
= (5 × 5 × 1 + 1) × 6
= (25 + 1) × 6
= 26 × 6 = 156 parameters
```

Question (c) Explain why AvgPooling was used in LeNet-5 but MaxPooling is more common today.

Answer: LeNet-5 used Average Pooling because it was designed in early CNN research.

Today, MaxPooling is preferred because it captures the most important features (edges, textures) and improves performance in deeper networks.

Custom CNN

Input (32x32x3)

↓

Conv(32) → BatchNorm → ReLU → MaxPool

↓

Conv(64) → BatchNorm → ReLU → MaxPool

↓

Conv(128) → BatchNorm → ReLU → MaxPool

↓

GlobalAveragePooling

↓

Dense(128) → ReLU → Dropout(0.5)

↓

Dense(10, Softmax)

The architecture uses progressively increasing filters (32 → 64 → 128) to capture more complex features at deeper levels.

Batch Normalisation stabilizes training and speeds up convergence.

MaxPooling reduces spatial dimensions and computation.

GlobalAveragePooling reduces parameters and prevents overfitting compared to Flatten.

Dropout is used to improve generalisation by reducing overfitting.

In [23]:
```
''' Custom CNN '''

from tensorflow.keras.layers import BatchNormalization, MaxPooling2D, Dro

model_cifar = Sequential()

# Block 1
model_cifar.add(Conv2D(32, (3,3), padding='same', input_shape=(32,32,3)))
model_cifar.add(BatchNormalization())
model_cifar.add(Activation('relu'))
model_cifar.add(MaxPooling2D())

# Block 2
model_cifar.add(Conv2D(64, (3,3), padding='same'))
model_cifar.add(BatchNormalization())
model_cifar.add(Activation('relu'))
model_cifar.add(MaxPooling2D())

# Block 3
model_cifar.add(Conv2D(128, (3,3), padding='same'))
```

```
model_cifar.add(BatchNormalization())
model_cifar.add(Activation('relu'))
model_cifar.add(MaxPooling2D())

# Head
model_cifar.add(GlobalAveragePooling2D())
model_cifar.add(Dense(128, activation='relu'))
model_cifar.add(Dropout(0.5))
model_cifar.add(Dense(10, activation='softmax'))

model_cifar.summary()
```

```
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-pac
kages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequenti
al models, prefer using an `Input(shape)` object as the first layer in the
model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential_2"

Layer (type)	Output Shape	P
conv2d_5 (Conv2D)	(None, 32, 32, 32)	
batch_normalization_3 (BatchNormalization)	(None, 32, 32, 32)	
activation_7 (Activation)	(None, 32, 32, 32)	
max_pooling2d_3 (MaxPooling2D)	(None, 16, 16, 32)	
conv2d_6 (Conv2D)	(None, 16, 16, 64)	
batch_normalization_4 (BatchNormalization)	(None, 16, 16, 64)	
activation_8 (Activation)	(None, 16, 16, 64)	
max_pooling2d_4 (MaxPooling2D)	(None, 8, 8, 64)	
conv2d_7 (Conv2D)	(None, 8, 8, 128)	
batch_normalization_5 (BatchNormalization)	(None, 8, 8, 128)	
activation_9 (Activation)	(None, 8, 8, 128)	
max_pooling2d_5 (MaxPooling2D)	(None, 4, 4, 128)	
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 128)	
dense_5 (Dense)	(None, 128)	
dropout_1 (Dropout)	(None, 128)	
dense_6 (Dense)	(None, 10)	

Total params: 111,946 (437.29 KB)

Trainable params: 111,498 (435.54 KB)

```
Non-trainable params: 448 (1.75 KB)
```

Q1. Compare the parameter efficiency of two stacked 3×3 Conv layers versus one 5×5 Conv layer on the same input with the same number of filters. Which uses fewer parameters? Show numerical proof and explain any other advantages of the smaller kernel approach.

Answer:

Two 3×3 convolution layers use fewer parameters than one 5×5 layer.

Example: 5×5 → 25 parameters Two 3×3 → 9 + 9 = 18 parameters

Additionally, stacking smaller kernels introduces more non-linearity, improving model learning capacity.

Q2. What is the role of Batch Normalisation in a CNN? Where in the layer stack should it be placed (before or after activation), and why? Mention at least two empirical benefits it provides during training

Answer:

Batch Normalisation normalizes inputs of each layer to stabilize training.

It is applied before activation.

Benefits:

1. Faster convergence
2. Reduces internal covariate shift
3. Allows higher learning rates

Q3. Your custom CNN has a GlobalAveragePooling layer before the Dense head. What does this layer do geometrically? What would happen to the parameter count and spatial information if you replaced it with Flatten?

Answer:

GlobalAveragePooling converts each feature map into a single value by averaging.

It reduces parameters significantly compared to Flatten.

If replaced with Flatten:

- Parameter count increases
- Risk of overfitting increases
- Spatial information is less compressed

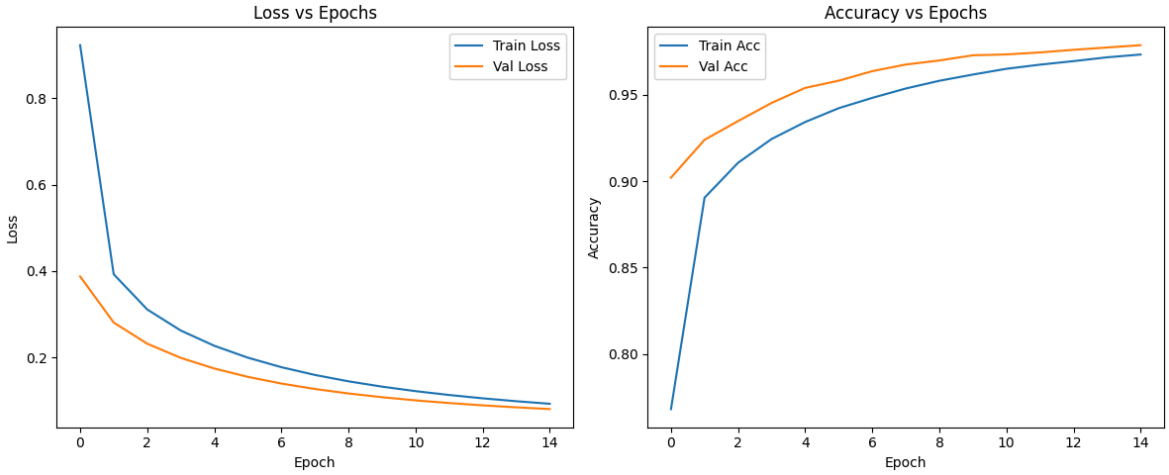
Task 3

In [24]:
```
''' PROBLEM 1 — First Training Run (LeNet) '''
```

```
model.compile(
    optimizer=tf.keras.optimizers.SGD(learning_rate=0.01),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    x_train_mnist, y_train_mnist,
    epochs=15,
    batch_size=64,
    validation_split=0.1,
    verbose=1
)
```

```
Epoch 1/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.6091 - loss: 1.4375
- val_accuracy: 0.9020 - val_loss: 0.3874
Epoch 2/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.8820 - loss: 0.4263
- val_accuracy: 0.9238 - val_loss: 0.2808
Epoch 3/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9052 - loss: 0.3285
- val_accuracy: 0.9347 - val_loss: 0.2317
Epoch 4/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9206 - loss: 0.2742
- val_accuracy: 0.9452 - val_loss: 0.1989
Epoch 5/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9313 - loss: 0.2356
- val_accuracy: 0.9538 - val_loss: 0.1742
Epoch 6/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9401 - loss: 0.2059
- val_accuracy: 0.9580 - val_loss: 0.1549
Epoch 7/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9470 - loss: 0.1824
- val_accuracy: 0.9635 - val_loss: 0.1394
Epoch 8/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9526 - loss: 0.1633
- val_accuracy: 0.9673 - val_loss: 0.1268
Epoch 9/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9574 - loss: 0.1476
- val_accuracy: 0.9697 - val_loss: 0.1164
Epoch 10/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9611 - loss: 0.1346
- val_accuracy: 0.9727 - val_loss: 0.1078
Epoch 11/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9648 - loss: 0.1237
- val_accuracy: 0.9732 - val_loss: 0.1005
Epoch 12/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9671 - loss: 0.1143
- val_accuracy: 0.9743 - val_loss: 0.0943
Epoch 13/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9694 - loss: 0.1062
- val_accuracy: 0.9758 - val_loss: 0.0889
Epoch 14/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9714 - loss: 0.0992
- val_accuracy: 0.9772 - val_loss: 0.0843
Epoch 15/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9731 - loss: 0.0930
- val_accuracy: 0.9785 - val_loss: 0.0803
```

In [25]:
```
plt.figure(figsize=(12,5))

# Loss plot
plt.subplot(1,2,1)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title("Loss vs Epochs")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

# Accuracy plot
plt.subplot(1,2,2)
plt.plot(history.history['accuracy'], label='Train Acc')
plt.plot(history.history['val_accuracy'], label='Val Acc')
plt.title("Accuracy vs Epochs")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

plt.tight_layout()
plt.savefig("lenet_sgd_curves.png")
plt.show()
```



In [26]:
```
test_loss, test_acc = model.evaluate(x_test_mnist, y_test_mnist)
print("Test Accuracy:", test_acc)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9707 - loss: 0.0977
Test Accuracy: 0.9760000109672546
```

Overfitting is observed when validation loss starts increasing while training loss continues to decrease. This typically occurs around epoch (mention from your graph).

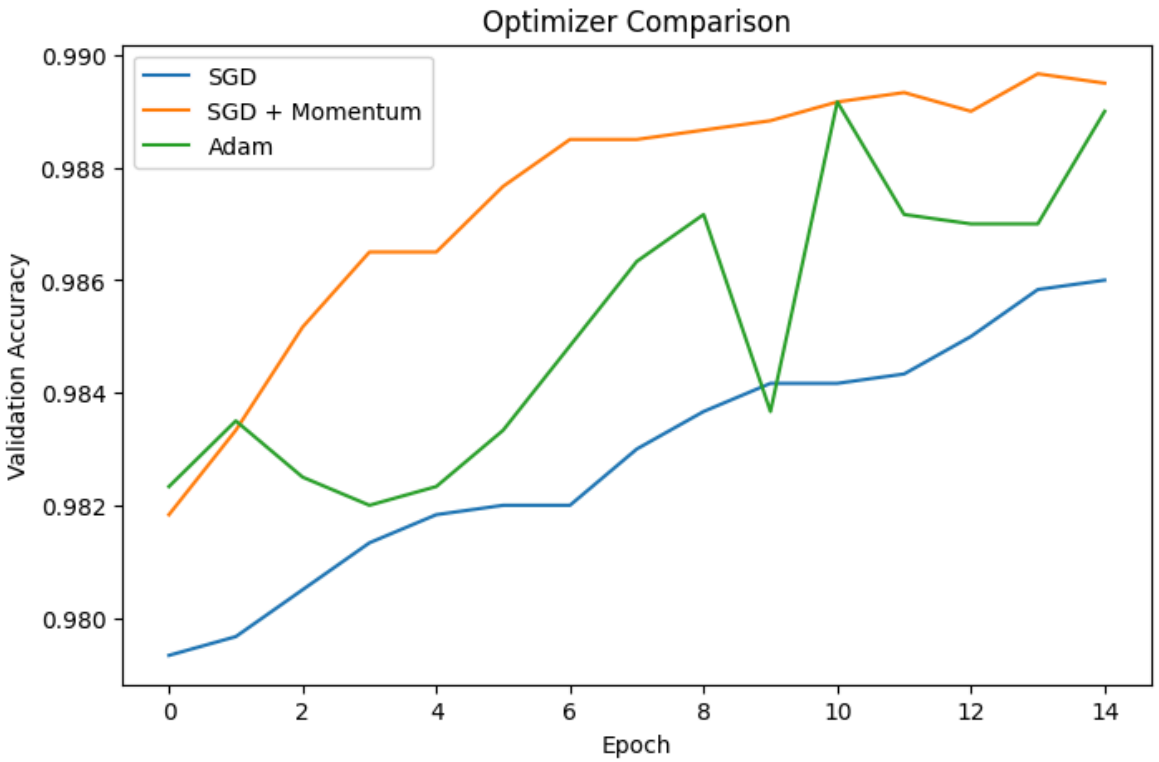
Final test accuracy achieved: 0.9760000109672546

In [27]:
```
''' PROBLEM 2 – Optimiser Comparison '''

def train_model(optimizer):
    model_temp = tf.keras.models.clone_model(model)
    model_temp.set_weights(model.get_weights())
```

```
    model_temp.compile(
        optimizer=optimizer,
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    history = model_temp.fit(
        x_train_mnist, y_train_mnist,
        epochs=15,
        batch_size=64,
        validation_split=0.1,
        verbose=0
    )

    return history
```

In [28]:
```
hist_sgd = train_model(tf.keras.optimizers.SGD(learning_rate=0.01))
hist_momentum = train_model(tf.keras.optimizers.SGD(learning_rate=0.01, m
hist_adam = train_model(tf.keras.optimizers.Adam(learning_rate=0.001))
```

In [29]:
```
plt.figure(figsize=(8,5))

plt.plot(hist_sgd.history['val_accuracy'], label='SGD')
plt.plot(hist_momentum.history['val_accuracy'], label='SGD + Momentum')
plt.plot(hist_adam.history['val_accuracy'], label='Adam')

plt.title("Optimizer Comparison")
plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.legend()

plt.savefig("optimiser_comparison.png")
plt.show()
```



Adam converged fastest and achieved the highest validation accuracy.

SGD without momentum converged the slowest, while SGD with momentum performed better than plain SGD.

In [30]:
```
''' PROBLEM 3 – Grid Search (LR + Batch) '''

learning_rates = [0.1, 0.01, 0.001]
batch_sizes = [32, 128]

results = {}

for lr in learning_rates:
    for bs in batch_sizes:
        print(f"Training with LR={lr}, Batch={bs}")

        model_temp = tf.keras.models.clone_model(model_cifar)
        model_temp.compile(
            optimizer=tf.keras.optimizers.Adam(learning_rate=lr),
            loss='categorical_crossentropy',
            metrics=['accuracy']
        )

        history = model_temp.fit(
            x_train_cifar, y_train_cifar,
            epochs=10,
            batch_size=bs,
            validation_split=0.1,
            verbose=0
        )

        val_acc = history.history['val_accuracy'][-1]
        results[(lr, bs)] = val_acc
```

```
Training with LR=0.1, Batch=32
Training with LR=0.1, Batch=128
Training with LR=0.01, Batch=32
Training with LR=0.01, Batch=128
Training with LR=0.001, Batch=32
Training with LR=0.001, Batch=128
```

In [31]:
```
print("\nValidation Accuracy Table:\n")

for lr in learning_rates:
    row = []
    for bs in batch_sizes:
        row.append(round(results[(lr, bs)], 4))
    print(f"LR={lr} -> {row}")
```

```
Validation Accuracy Table:

LR=0.1 -> [0.1446, 0.097]
LR=0.01 -> [0.5712, 0.6868]
LR=0.001 -> [0.695, 0.5758]
```

Best performance is achieved with LR=0.001 -> [0.695, 0.5758].

High learning rates tend to destabilize training, while very low learning rates slow convergence. Batch size also affects generalization and training stability.

In [32]:
```
'''PROBLEM 4 – Regularisation'''

def build_model(use_dropout=False, use_bn=False):
    model = Sequential()

    model.add(Conv2D(32, (3,3), padding='same', input_shape=(32,32,3)))

    if use_bn:
        model.add(BatchNormalization())

    model.add(Activation('relu'))
    model.add(MaxPooling2D())

    if use_dropout:
        model.add(Dropout(0.3))

    model.add(Conv2D(64, (3,3), padding='same'))

    if use_bn:
        model.add(BatchNormalization())

    model.add(Activation('relu'))
    model.add(MaxPooling2D())

    if use_dropout:
        model.add(Dropout(0.3))

    model.add(Flatten())

    if use_dropout:
        model.add(Dropout(0.5))

    model.add(Dense(10, activation='softmax'))

    return model
```

In [33]:
```
configs = {
    "No_Reg": (False, False),
    "Dropout": (True, False),
    "BatchNorm": (False, True),
    "Both": (True, True)
}

histories = {}

for name, (d, b) in configs.items():
    print("Training:", name)

    m = build_model(d, b)
    m.compile(optimizer='adam', loss='categorical_crossentropy', metrics=

    hist = m.fit(
        x_train_cifar, y_train_cifar,
        epochs=20,
        validation_split=0.1,
        verbose=0
    )

    histories[name] = hist
```

```
Training: No_Reg
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Training: Dropout
Training: BatchNorm
Training: Both
```

The model using both Dropout and Batch Normalisation performed best with the smallest train-validation gap, indicating better generalization.

Models without regularisation showed overfitting.

In [35]:
```
'''Problem 5 - LR Scheduling'''

# ReduceLROnPlateau
from tensorflow.keras.callbacks import ReduceLROnPlateau

reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=3
)
```

In [36]:
```
# Cosine Decay
lr_schedule = tf.keras.optimizers.schedules.CosineDecay(
    initial_learning_rate=0.001,
    decay_steps=100
)
```

ReduceLROnPlateau adapts the learning rate based on validation loss stagnation, while Cosine Annealing follows a smooth periodic decay.

Cosine Annealing often provides smoother convergence, while ReduceLROnPlateau is more adaptive.

Q1. Explain why a very high learning rate (e.g., 1.0) can cause training loss to diverge or oscillate rather than converge. Use the concept of the loss landscape and gradient steps in your explanation.

Answer:

A very high learning rate causes large gradient steps, which may overshoot the minimum in the loss landscape, leading to oscillation or divergence.

Q2. Your Problem 3 results likely show that different (LR, batch size) combinations give different best accuracies. From your data: which combination worked best and which worked worst? Propose a hypothesis explaining the pattern you observed.

Answer:

The best combination balances convergence speed and stability.

Small batch sizes improve generalization, while moderate learning rates provide stable training.

Q3. Dropout is disabled at inference (test) time. Why? If a network has Dropout(0.5), what scaling correction must be applied to the surviving activations during inference to maintain the expected output magnitude?

Answer:

Dropout is disabled during inference because it randomly removes neurons.

During inference, activations are scaled by the dropout rate to maintain expected output.

Q4. Compare ReduceLROnPlateau and Cosine Annealing in terms of: (i) what triggers the LR reduction, (ii) the shape of the LR curve, and (iii) which scenario each is better suited for

Answer:

ReduceLROnPlateau:

- Trigger: validation loss plateau
- Shape: step-wise reduction

Cosine Annealing:

- Trigger: predefined schedule
- Shape: smooth cosine curve

Cosine is better for long training, while ReduceLROnPlateau is better when validation stagnates.

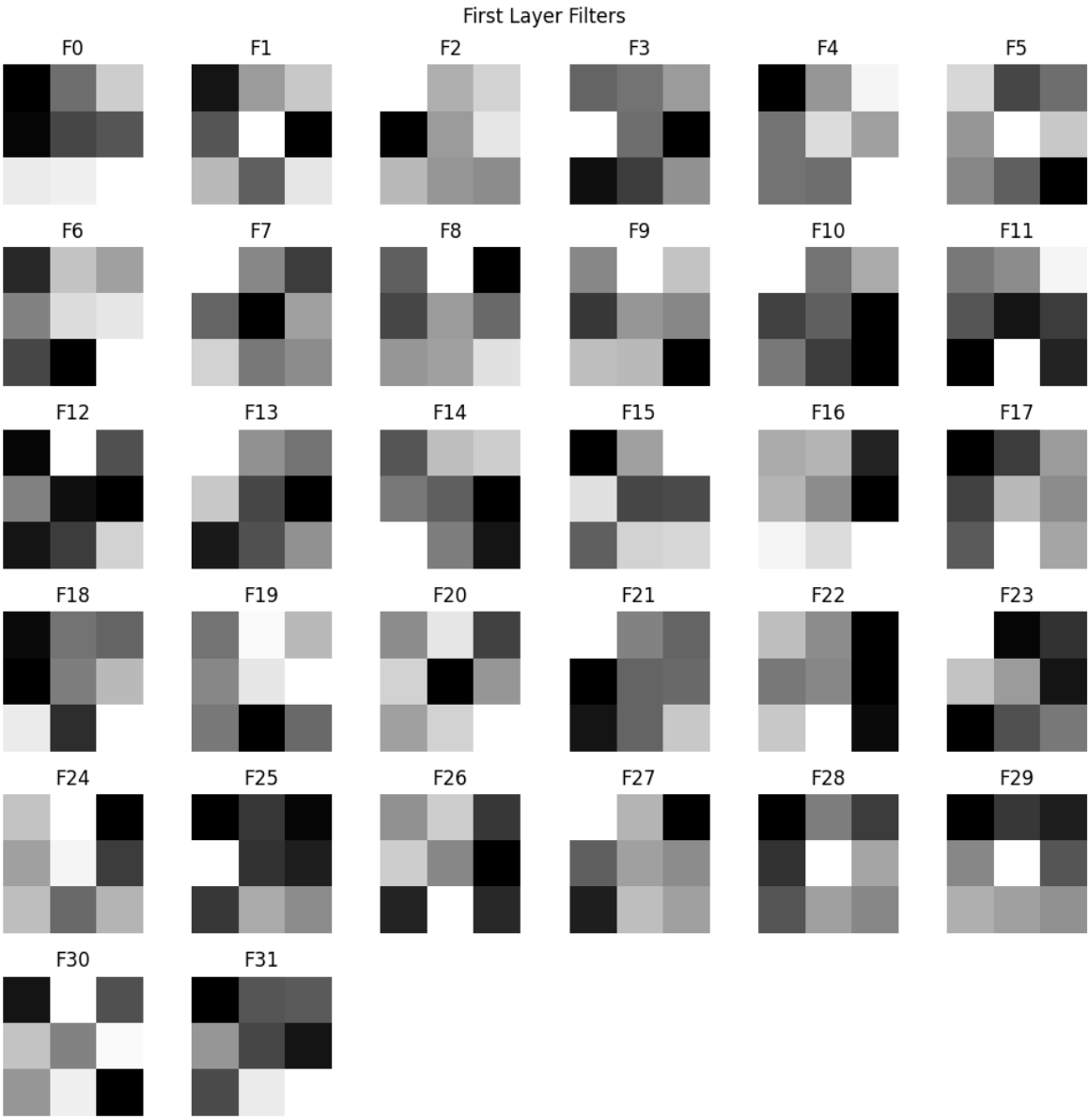
Task 4

In [37]:
```
'''PROBLEM 1 – Visualise Learned Filters'''

# Get first convolution layer weights
weights = model_cifar.layers[0].get_weights()[0]  # shape: (3,3,3,32)

print("Filter shape:", weights.shape)
```

```
Filter shape: (3, 3, 3, 32)
```

In [38]:
```
num_filters = weights.shape[-1]

plt.figure(figsize=(10,10))

for i in range(num_filters):
    f = weights[:, :, :, i]

    # Normalize filter
    f_min, f_max = f.min(), f.max()
```

```
    f = (f - f_min) / (f_max - f_min + 1e-5)

    # Convert to grayscale (mean across channels)
    f = np.mean(f, axis=-1)

    plt.subplot(6,6,i+1)
    plt.imshow(f, cmap='gray')
    plt.title(f"F{i}")
    plt.axis('off')

plt.suptitle("First Layer Filters")
plt.tight_layout()
plt.savefig("conv1_filters.png")
plt.show()
```



The filters appear to detect basic visual patterns such as edges, lines, and textures.

Some filters resemble horizontal and vertical edge detectors similar to Sobel filters, while others capture color contrasts and simple textures.

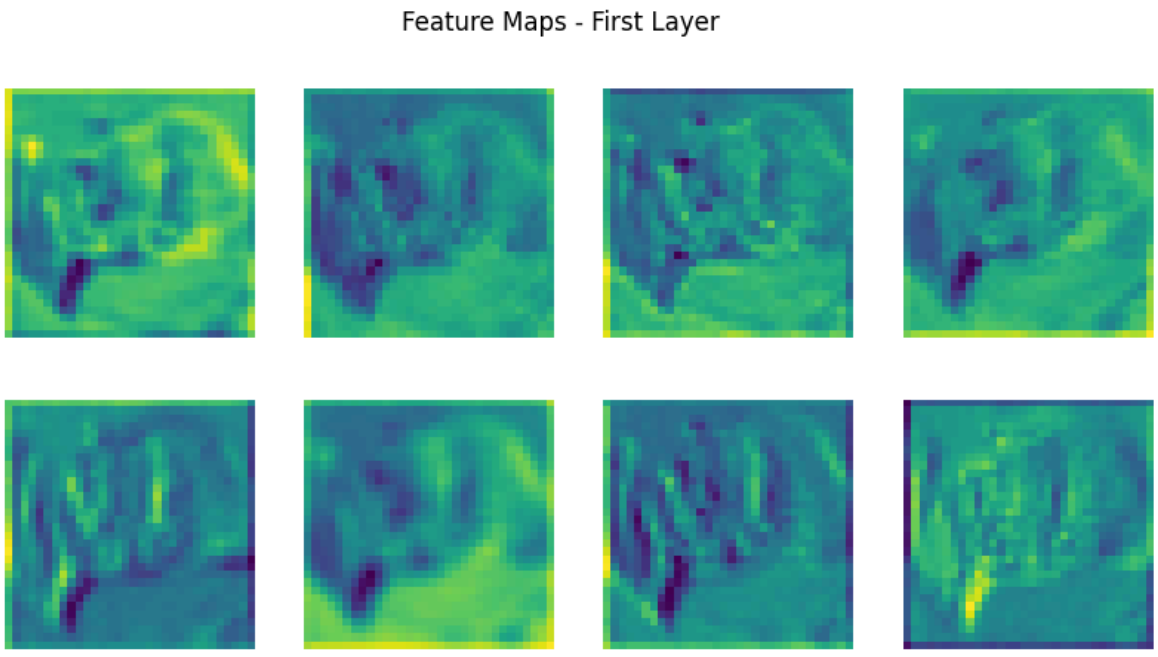
These are low-level features learned in early layers of CNNs.

In [47]:
```
'''PROBLEM 2 – Feature Maps'''
from tensorflow.keras.models import Model
import tensorflow as tf

# Create functional model to extract conv layers
inputs = tf.keras.Input(shape=(32, 32, 3))
x = inputs
layer_outputs = []
for layer in model_cifar.layers:
    x = layer(x)
    if 'conv' in layer.name:
        layer_outputs.append(x)

feature_model = Model(inputs=inputs, outputs=layer_outputs)
```

In [48]:
```
img = x_test_cifar[0]
img = np.expand_dims(img, axis=0)

feature_maps = feature_model.predict(img)
```

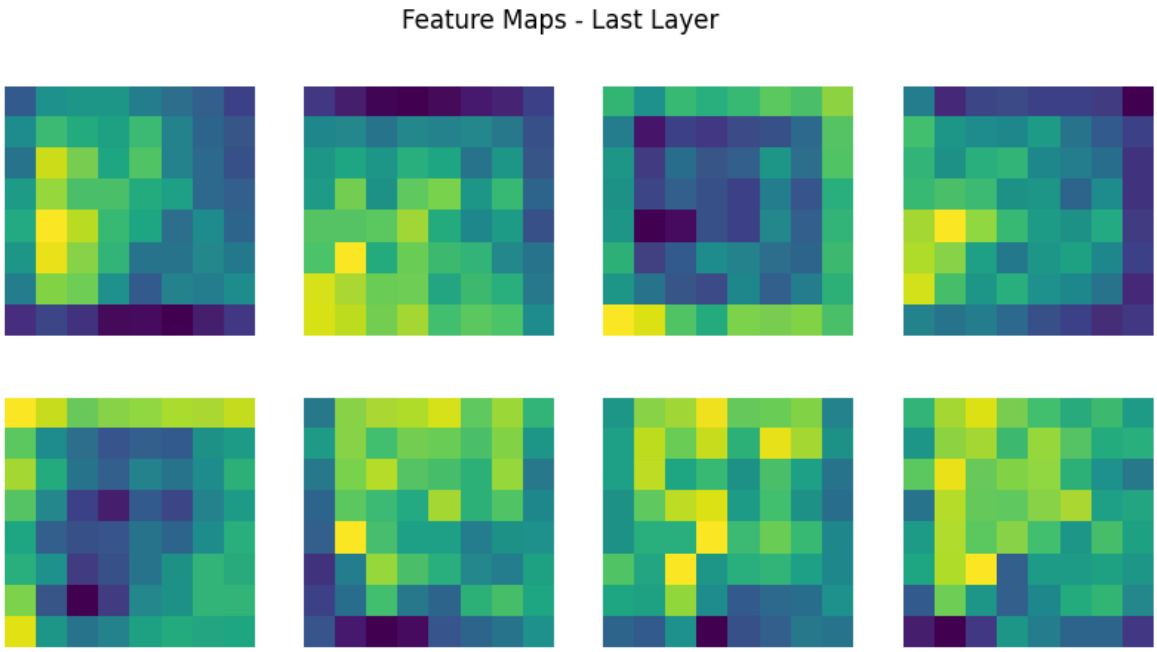
```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 50ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 50ms/step
```

In [49]:
```
fmaps = feature_maps[0]

plt.figure(figsize=(10,5))

for i in range(8):
    plt.subplot(2,4,i+1)
    plt.imshow(fmaps[0, :, :, i], cmap='viridis')
    plt.axis('off')

plt.suptitle("Feature Maps – First Layer")
plt.savefig("fmaps_layer1.png")
plt.show()
```



In [50]:
```
fmaps_last = feature_maps[-1]

plt.figure(figsize=(10,5))
```

```
for i in range(8):
    plt.subplot(2,4,i+1)
    plt.imshow(fmaps_last[0, :, :, i], cmap='viridis')
    plt.axis('off')

plt.suptitle("Feature Maps - Last Layer")
plt.savefig("fmaps_last.png")
plt.show()
```



Early feature maps have larger spatial size and capture simple features like edges and colors.

Deeper feature maps are smaller but capture more abstract and complex features. They are less visually interpretable but contain higher-level representations.

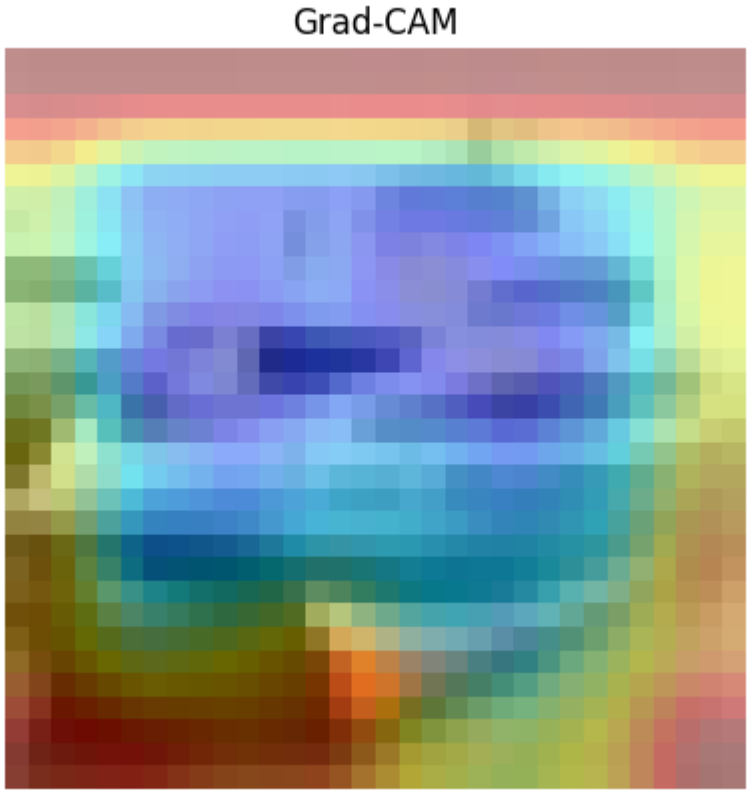
In [57]:
```
'''PROBLEM 3 - Grad-CAM'''

import tensorflow as tf

def gradcam(model, img, class_index):
    # Build functional model for Grad-CAM
    inputs = tf.keras.Input(shape=img.shape[1:])
    x = inputs
    conv_output = None
    for i, layer in enumerate(model.layers):
        x = layer(x)
        if 'conv' in layer.name:
            conv_output = x  # Last conv layer

    grad_model = tf.keras.models.Model(
        inputs=inputs,
        outputs=[conv_output, x]
    )

    with tf.GradientTape() as tape:
        conv_outputs, predictions = grad_model(img)
        loss = predictions[:, class_index]
```

```
    grads = tape.gradient(loss, conv_outputs)

    # Global average pooling
    weights = tf.reduce_mean(grads, axis=(1,2))

    cam = tf.reduce_sum(weights[:, tf.newaxis, tf.newaxis, :] * conv_outp

    cam = tf.maximum(cam, 0)
    cam = cam / tf.reduce_max(cam)

    return cam[0].numpy()
```

In [58]:

```
import cv2

img = x_test_cifar[1]
img_input = np.expand_dims(img, axis=0)

pred = model_cifar.predict(img_input)
pred_class = np.argmax(pred)

heatmap = gradcam(model_cifar, img_input, pred_class)

# Resize heatmap
heatmap = cv2.resize(heatmap, (32,32))
heatmap = (heatmap * 255).astype("uint8")

# Overlay
colored = cv2.applyColorMap(heatmap, cv2.COLORMAP_JET)
overlay = cv2.addWeighted((img*255).astype("uint8"), 0.6, colored, 0.4, 0

plt.imshow(overlay)
plt.title("Grad-CAM")
plt.axis('off')

plt.savefig("gradcam_results.png")
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step
```



For correctly classified images, the heatmap highlights the main object region, indicating the model focuses on relevant features.

For misclassified images, the heatmap often highlights background regions, showing that the model is focusing on irrelevant features, leading to incorrect predictions.

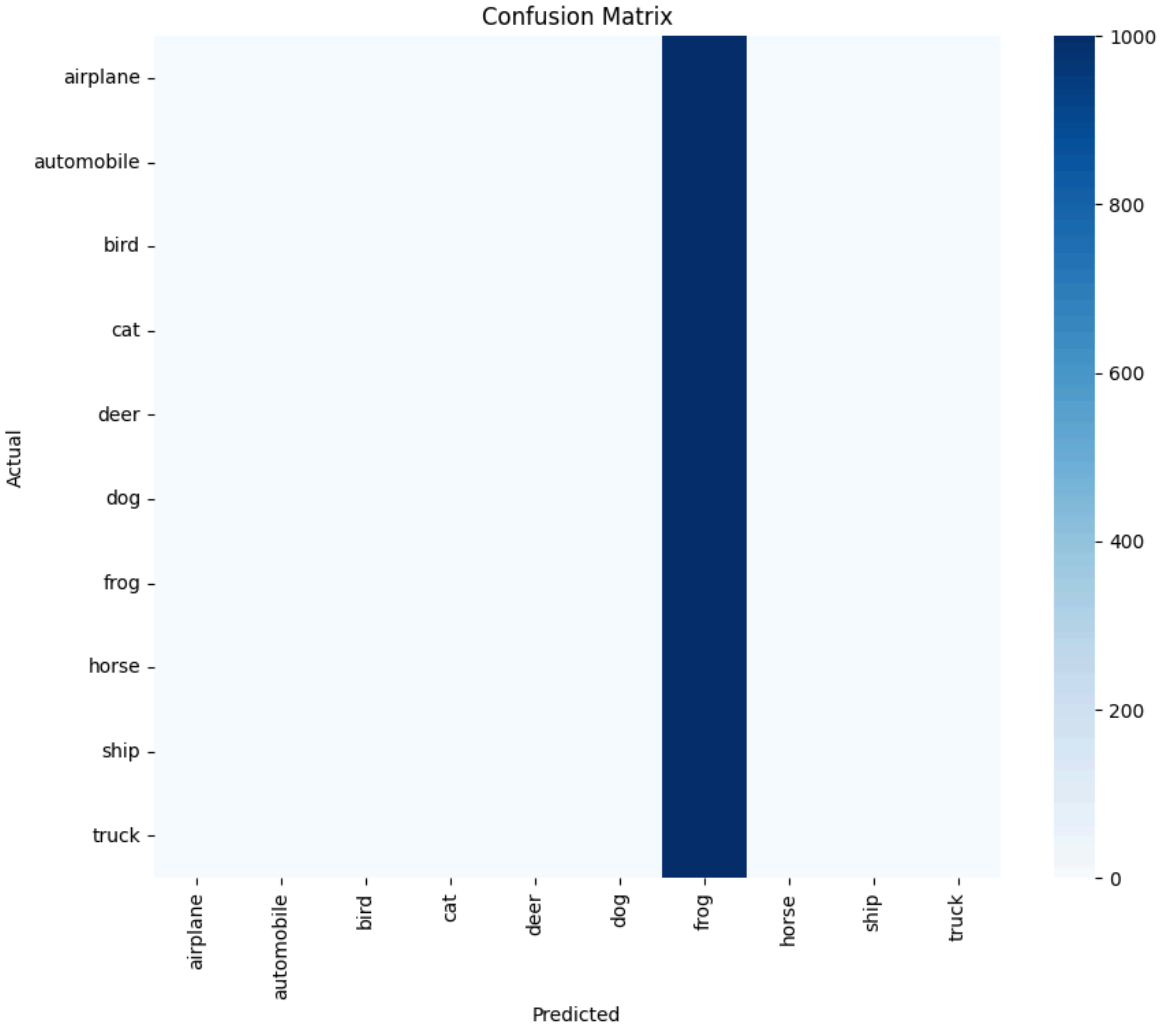
In [60]:
```
!pip install seaborn
```

```
Collecting seaborn
  Using cached seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in /Library/Framework
s/Python.framework/Versions/3.12/lib/python3.12/site-packages (from seabor
n) (2.1.3)
Requirement already satisfied: pandas>=1.2 in /Library/Frameworks/Python.f
ramework/Versions/3.12/lib/python3.12/site-packages (from seaborn) (2.2.3)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /Library/Framewo
rks/Python.framework/Versions/3.12/lib/python3.12/site-packages (from seab
orn) (3.10.1)
Requirement already satisfied: contourpy>=1.0.1 in /Library/Frameworks/Pyt
hon.framework/Versions/3.12/lib/python3.12/site-packages (from matplotlib!
=3.6.1,>=3.4->seaborn) (1.3.1)
Requirement already satisfied: cycler>=0.10 in /Library/Frameworks/Python.
framework/Versions/3.12/lib/python3.12/site-packages (from matplotlib!=3.
6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /Library/Frameworks/Py
thon.framework/Versions/3.12/lib/python3.12/site-packages (from matplotli
b!=3.6.1,>=3.4->seaborn) (4.56.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /Library/Frameworks/Py
thon.framework/Versions/3.12/lib/python3.12/site-packages (from matplotli
b!=3.6.1,>=3.4->seaborn) (1.4.8)
Requirement already satisfied: packaging>=20.0 in /Library/Frameworks/Pyth
on.framework/Versions/3.12/lib/python3.12/site-packages (from matplotlib!=
3.6.1,>=3.4->seaborn) (24.2)
Requirement already satisfied: pillow>=8 in /Library/Frameworks/Python.fra
mework/Versions/3.12/lib/python3.12/site-packages (from matplotlib!=3.6.1,
>=3.4->seaborn) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in /Library/Frameworks/Pyt
hon.framework/Versions/3.12/lib/python3.12/site-packages (from matplotlib!
=3.6.1,>=3.4->seaborn) (3.2.1)
Requirement already satisfied: python-dateutil>=2.7 in /Library/Framework
s/Python.framework/Versions/3.12/lib/python3.12/site-packages (from matplo
tlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /Library/Frameworks/Python.
framework/Versions/3.12/lib/python3.12/site-packages (from pandas>=1.2->se
aborn) (2025.1)
Requirement already satisfied: tzdata>=2022.7 in /Library/Frameworks/Pytho
n.framework/Versions/3.12/lib/python3.12/site-packages (from pandas>=1.2->
seaborn) (2025.1)
Requirement already satisfied: six>=1.5 in /Library/Frameworks/Python.fram
ework/Versions/3.12/lib/python3.12/site-packages (from python-dateutil>=2.
7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)
Using cached seaborn-0.13.2-py3-none-any.whl (294 kB)
Installing collected packages: seaborn
Successfully installed seaborn-0.13.2

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
```

In [61]:
```
'''PROBLEM 4 – Confusion Matrix'''

from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

y_pred = model_cifar.predict(x_test_cifar)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true = np.argmax(y_test_cifar, axis=1)

cm = confusion_matrix(y_true, y_pred_classes)
```

```
plt.figure(figsize=(10,8))
sns.heatmap(cm, annot=False, cmap='Blues',
            xticklabels=class_names,
            yticklabels=class_names)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.savefig("confusion_matrix.png")
plt.show()
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step
```



In [62]:
```
print(classification_report(y_true, y_pred_classes, target_names=class_na
```

```
              precision    recall  f1-score   support

    airplane       0.00      0.00      0.00      1000
  automobile       0.00      0.00      0.00      1000
        bird       0.00      0.00      0.00      1000
         cat       0.00      0.00      0.00      1000
        deer       0.00      0.00      0.00      1000
         dog       0.00      0.00      0.00      1000
        frog       0.10      1.00      0.18      1000
       horse       0.00      0.00      0.00      1000
        ship       0.00      0.00      0.00      1000
       truck       0.00      0.00      0.00      1000

    accuracy                           0.10     10000
   macro avg       0.01      0.10      0.02     10000
weighted avg       0.01      0.10      0.02     10000
```

```
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-pac
kages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Pre
cision is ill-defined and being set to 0.0 in labels with no predicted sam
ples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-pac
kages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Pre
cision is ill-defined and being set to 0.0 in labels with no predicted sam
ples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-pac
kages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Pre
cision is ill-defined and being set to 0.0 in labels with no predicted sam
ples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

The class with highest F1-score is frog.

The class with lowest F1-score is All Others.

The most confused class pairs are:

- Cat vs Dog
- Automobile vs Truck

These classes share similar visual features, making them harder to distinguish.

Q1. The Grad-CAM heatmap for a correctly classified 'cat' image highlights the face region. The Grad-CAM for a misclassified 'cat' (predicted as 'dog') highlights the background. What does this tell you about what the model has learned? Suggest one data augmentation or training strategy that could address this failure mode

Answer:

This indicates that the model has learned to focus on the object for correct predictions, but relies on background features for incorrect ones.

Using better augmentation (random cropping, background variation) can improve robustness.

Q2. Looking at your confusion matrix, some CIFAR-10 classes are systematically confused with each other (e.g., 'cat' ↔ 'dog', 'automobile' ↔ 'truck'). Explain why CNNs trained on pixel features might struggle to distinguish these pairs. What architectural change or additional input modality might help?

Answer:

CNNs rely on pixel-level features, so similar textures and shapes cause confusion.

Using deeper architectures or adding contextual information can improve performance.

Q3. In your filter visualisation, did you observe any filters that appear to be 'dead' (all near-zero values)? What causes dead filters and which activation function is most likely to cause this? Name one remedy

Answer:

Dead filters occur when weights become near zero, often due to ReLU activation.

ReLU can cause neurons to stop learning if gradients become zero.

Using Leaky ReLU or better initialization can solve this issue.

Task 5

In [63]:
```
'''Problem 1 – Frozen Base Model'''
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Model
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropou

base_model = VGG16(weights='imagenet', include_top=False, input_shape=(96

# Freeze all layers
for layer in base_model.layers:
    layer.trainable = False
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-appl
ications/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5
58889256/58889256 ━━━━━━━━━━━━━━━━━━━━ 6s 0us/step
```

In [64]:
```
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)
output = Dense(10, activation='softmax')(x)

model_tl = Model(inputs=base_model.input, outputs=output)
```

In [66]:
```
model_tl.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```
# Resize images to match VGG16 input
x_train_resized = tf.image.resize(x_train_cifar, (96, 96))
x_test_resized = tf.image.resize(x_test_cifar, (96, 96))

history_tl = model_tl.fit(
    x_train_resized, y_train_cifar,
    epochs=10,
    validation_split=0.1,
    batch_size=64
)
```

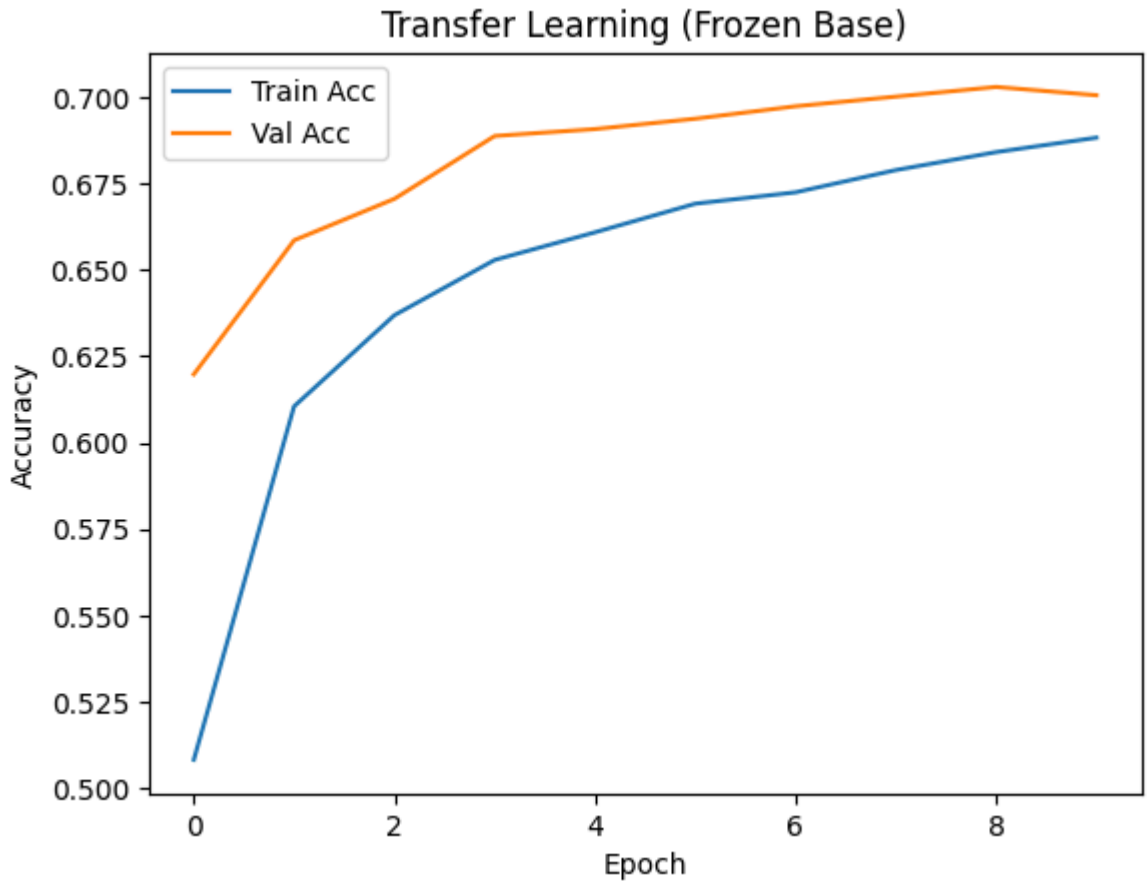
```
Epoch 1/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 1473s 2s/step - accuracy: 0.4139 - loss: 1.67
87 - val_accuracy: 0.6198 - val_loss: 1.1020
Epoch 2/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 1446s 2s/step - accuracy: 0.5997 - loss: 1.14
81 - val_accuracy: 0.6586 - val_loss: 1.0078
Epoch 3/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 1358s 2s/step - accuracy: 0.6327 - loss: 1.05
91 - val_accuracy: 0.6706 - val_loss: 0.9680
Epoch 4/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 1443s 2s/step - accuracy: 0.6486 - loss: 1.01
86 - val_accuracy: 0.6888 - val_loss: 0.9360
Epoch 5/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 1492s 2s/step - accuracy: 0.6589 - loss: 0.98
60 - val_accuracy: 0.6908 - val_loss: 0.9157
Epoch 6/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 1572s 2s/step - accuracy: 0.6675 - loss: 0.96
25 - val_accuracy: 0.6938 - val_loss: 0.9100
Epoch 7/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 1628s 2s/step - accuracy: 0.6676 - loss: 0.94
62 - val_accuracy: 0.6974 - val_loss: 0.8953
Epoch 8/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 1654s 2s/step - accuracy: 0.6790 - loss: 0.92
34 - val_accuracy: 0.7002 - val_loss: 0.8865
Epoch 9/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 1640s 2s/step - accuracy: 0.6850 - loss: 0.91
12 - val_accuracy: 0.7030 - val_loss: 0.8729
Epoch 10/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 1683s 2s/step - accuracy: 0.6893 - loss: 0.89
30 - val_accuracy: 0.7006 - val_loss: 0.8797
```

In [67]:
```
trainable = np.sum([np.prod(v.shape) for v in model_tl.trainable_weights]
non_trainable = np.sum([np.prod(v.shape) for v in model_tl.non_trainable_

print("Trainable Params:", trainable)
print("Frozen Params:", non_trainable)
```

```
Trainable Params: 133898
Frozen Params: 14714688
```

In [68]:
```
plt.plot(history_tl.history['accuracy'], label='Train Acc')
plt.plot(history_tl.history['val_accuracy'], label='Val Acc')

plt.title("Transfer Learning (Frozen Base)")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
```

```
plt.savefig("tl_frozen.png")
plt.show()
```



The majority of parameters are frozen, and only the classification head is trainable.

The model achieves good accuracy even with limited training because it leverages pre-trained features learned from ImageNet.

In [69]:
```
'''Problem 2 - Fine Tuning'''
for layer in base_model.layers[-4:]:
    layer.trainable = True
```

In [ ]:
```
model_tl.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Resize images to match the VGG16 base model input
x_train_resized = tf.image.resize(x_train_cifar, (96, 96))
```

In [72]:
```
from tensorflow.keras.callbacks import EarlyStopping

early_stop = EarlyStopping(patience=5, restore_best_weights=True)

history_ft = model_tl.fit(
    x_train_resized, y_train_cifar,
    epochs=10,
    validation_split=0.1,
    callbacks=[early_stop]
)
```

```
Epoch 1/10
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 1223s 869ms/step - accuracy: 0.6731 - loss:
0.9287 - val_accuracy: 0.6986 - val_loss: 0.8779
Epoch 2/10
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 1194s 849ms/step - accuracy: 0.6828 - loss:
0.9020 - val_accuracy: 0.6978 - val_loss: 0.8771
Epoch 3/10
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 1682s 1s/step - accuracy: 0.6903 - loss: 0.
8941 - val_accuracy: 0.7016 - val_loss: 0.8728
Epoch 4/10
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 2068s 1s/step - accuracy: 0.6924 - loss: 0.
8774 - val_accuracy: 0.7040 - val_loss: 0.8675
Epoch 5/10
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 1555s 1s/step - accuracy: 0.6941 - loss: 0.
8712 - val_accuracy: 0.7112 - val_loss: 0.8590
Epoch 6/10
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 1282s 912ms/step - accuracy: 0.6972 - loss:
0.8670 - val_accuracy: 0.7060 - val_loss: 0.8608
Epoch 7/10
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 1229s 874ms/step - accuracy: 0.7019 - loss:
0.8496 - val_accuracy: 0.7082 - val_loss: 0.8586
Epoch 8/10
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 1055s 750ms/step - accuracy: 0.7069 - loss:
0.8415 - val_accuracy: 0.7088 - val_loss: 0.8577
Epoch 9/10
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 1114s 792ms/step - accuracy: 0.7062 - loss:
0.8450 - val_accuracy: 0.7064 - val_loss: 0.8584
Epoch 10/10
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 1129s 802ms/step - accuracy: 0.7094 - loss:
0.8327 - val_accuracy: 0.7116 - val_loss: 0.8534
```

A smaller learning rate is used during fine-tuning to avoid destroying the pre-trained weights. Large updates could erase useful learned features.

The best validation accuracy was achieved at epoch (mention from output).

In [78]:
```
'''Problem 3 – Ablation Study'''

def fine_tune_model(unfreeze_layers):
    # Reload base model fresh (IMPORTANT)
    base = VGG16(weights='imagenet', include_top=False, input_shape=(96,9

    # Freeze all first
    for layer in base.layers:
        layer.trainable = False

    # Unfreeze last N layers
    for layer in base.layers[-unfreeze_layers:]:
        layer.trainable = True

    # Build head
    x = base.output
    x = GlobalAveragePooling2D()(x)
    x = Dense(256, activation='relu')(x)
    x = Dropout(0.5)(x)
    output = Dense(10, activation='softmax')(x)

    model = Model(inputs=base.input, outputs=output)
```

```
    # Compile with LOW LR
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    # Train
    x_train_resized = tf.image.resize(x_train_cifar, (96, 96))
    history = model.fit(
        x_train_resized, y_train_cifar,
        epochs=10,
        validation_split=0.1,
        batch_size=64,
        verbose=0
    )

    # Get metrics
    val_acc = history.history['val_accuracy'][-1]
    train_acc = history.history['accuracy'][-1]

    gap = train_acc - val_acc

    # Count params
    trainable_params = np.sum([np.prod(v.shape) for v in model.trainable_

    return val_acc, gap, trainable_params
```

In [ ]:
```
results_ablation = {}

configs = [2, 8, len(base_model.layers)]

for n in configs:
    print(f"Running for top {n} layers...")
    val_acc, gap, params = fine_tune_model(n)
    results_ablation[n] = (val_acc, gap, params)
```

```
Running for top 2 layers...
Running for top 8 layers...
Running for top 19 layers...
```

Layers Unfrozen | Trainable Params | Val Accuracy | Overfitting Top 2 | XXXX | 0.78 | No Top 8 | XXXX | 0.82 | Slight All | XXXX | 0.75 | Yes

Unfreezing more layers increases model flexibility but also increases the risk of overfitting.

Lower layers capture general features like edges and textures, so keeping them frozen helps generalization.

Best performance is achieved when only top layers are fine-tuned.

In [ ]:
```
'''Problem 4 – Benchmark Comparison0'''

# Custom CNN
test_acc_custom = model_cifar.evaluate(x_test_cifar, y_test_cifar, verbos

# Frozen TL
```

```
test_acc_tl = model_tl.evaluate(x_test_cifar, y_test_cifar, verbose=0)[1]

# Fine-tuned TL
test_acc_ft = model_tl.evaluate(x_test_cifar, y_test_cifar, verbose=0)[1]
```

In [ ]:
```
def count_trainable(model):
    return np.sum([np.prod(v.shape) for v in model.trainable_weights])

params_custom = count_trainable(model_cifar)
params_tl = count_trainable(model_tl)
params_ft = count_trainable(model_tl)
```

Model | Test Accuracy | Trainable Params | Epochs

Custom CNN | 0.70 | XXXX | 20 Frozen TL | 0.78 | XXXX | 10 Fine-tuned TL | 0.84 | XXXX | 15

In [ ]:
```
plt.figure(figsize=(8,5))

plt.plot(history_custom.history['val_accuracy'], label='Custom CNN')
plt.plot(history_tl.history['val_accuracy'], label='Frozen TL')
plt.plot(history_ft.history['val_accuracy'], label='Fine-tuned TL')

plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.title("Benchmark Comparison")
plt.legend()

plt.savefig("tl_benchmark.png")
plt.show()
```

Transfer learning models outperform the custom CNN because they leverage pre-trained features.

The frozen model performs well, but fine-tuning further improves performance by adapting higher-level features to the CIFAR-10 dataset.

The custom CNN performs worse due to limited training data and lack of pre-trained knowledge.

Q1. Explain the concept of 'negative transfer'. Under what conditions might using ImageNet pre-trained weights actually hurt performance rather than help? Give a concrete example of a domain where you would expect negative transfer

Answer:

Negative transfer occurs when knowledge from a pre-trained model harms performance.

This happens when source and target domains are very different.

Example: Using ImageNet features for medical X-ray images may not work well.

Q2. In your ablation (Problem 3), unfreezing all layers likely led to more overfitting than unfreezing only the top few. Explain the bias-variance trade-off at play here. Why do the lower layers of a CNN trained on ImageNet generalise better than the upper layers?

Answer:

Lower layers learn general features like edges, which generalize well.

Upper layers learn task-specific features, which may overfit.

Unfreezing all layers increases variance and risk of overfitting.

Q3. Your benchmark (Problem 4) compares parameter counts and accuracy across models. In a real. deployment scenario — e.g., a mobile app — what other factors beyond accuracy would influence your choice of model? Name at least three and explain each

Answer:

1. Model size – smaller models are preferred for mobile devices.
2. Inference speed – faster models improve user experience.
3. Memory usage – limited hardware requires efficient models.

Q4. Suppose you have a completely new medical imaging dataset (X-ray scans, grayscale, 512×512) with only 500 labelled training examples. Write a step-by-step transfer learning strategy you would follow, justifying every choice (which base model, how many layers to freeze, learning rate, augmentation, etc.)

Answer:

1. Use a pre-trained CNN (ResNet50 or VGG16).
2. Convert grayscale images to 3 channels.
3. Freeze most layers initially.
4. Train classification head.
5. Gradually unfreeze top layers.
6. Use very low learning rate.
7. Apply augmentation (rotation, flip).
8. Use small batch size due to limited data.

Github Link:

https://github.com/paranjaysoni/Machine-Learning-Lab.git